1 Mutant Generation

MutPy has been used in several earlier studies. However, the tool is no longer maintained and, in particular, does not support Python versions beyond 3.8. For older Python versions, MutPy can still be used if only the generation of mutants is required. When reproducibility of mutants is also needed, MutPy relies on unparsing the AST via the `astmonkey` package, which itself supports only Python up to version 3.3. Consequently, attempting to reproduce mutants may fail for Python versions in the range 3.4–3.8, even if mutant generation itself succeeds. Furthermore, due to changes introduced in Python over time (for example, updates to the `collections` module), there is no guarantee that MutPy's mutant generation behaves correctly in these newer versions.

It is worth noting that Python 3.9 introduced an `unparse` function in the standard library. In principle, MutPy could be modified to use this built-in function instead of the `astmonkey` unparsing facilities. However, such a change would allow reproducibility only for Python versions 3.9 and above, and would not resolve issues for versions 3.4–3.8. This limitation is particularly relevant given that many projects in BugsInPy fall exactly within this version range.

CodeMutate is a tool in early development by Cédric that addresses these issues by replacing MutPy's AST handling with `tree-sitter`, which provides a uniform and version-independent Python AST. As a result, CodeMutate enables both generation and reproducibility of mutants across the relevant Python versions, and is expected to remain compatible with future Python releases as well.

It is also straightforward to modify CodeMutate's source code so that it produces the meta-information required for latent-mutant analysis. Such metadata can be generated with only minor adjustments and can be made compatible with the input format expected by the plugin discussed later.

The downsides of using CodeMutate are twofold. First, as with any early-stage tool, there is no guarantee that it is free of bugs. Second, it remains an open question whether all mutation operators precisely mirror MutPy's original semantics. On the positive side, the tool is not a complete reimplementation but rather a direct transfer of MutPy's operator logic to `tree-sitter`, which reduces the likelihood of semantic mismatches. Nonetheless, the absence of a thorough validation step means that complete correctness cannot be assumed, although CodeMutate has passed our tests on a number of projects so far.

2 Mutation Execution

Current tooling within the Python ecosystem can be used to execute mutants and record their results when only a single version of a project is of interest. However, in a software-evolution setting—for example, when propagated mutants must be executed on commits other than the one in which they were originally introduced—existing tools do not, to the best of our knowledge, provide functionality for taking arbitrary mutants as input and executing or analysing

them accordingly. In addition to this main limitation, practical issues such as MutPy failures on a few tested projects might further restrict the applicability of existing tools.

For these reasons, we implemented two components, `pytest-mutation-runner` and `mutant-sandbox-runner.py`, which together enable, to a good degree of efficiency, the execution of programs and their corresponding mutants against a test suite, while also collecting and storing execution results for subsequent analysis.

2.1 pytest-mutation-runner

The component `pytest-mutation-runner` is a lightweight Pytest plugin that can be installed via:

```
pip install -e .
```

after cloning its repository. The plugin offers two mutually exclusive execution modes: *program* and *mutant*.

2.1.1 Program mode

In program mode, the plugin executes the original (unmutated) Python program against a user-specified test suite, provided in the same manner as for standard Pytest execution. During the run, the plugin records information such as pass/fail outcomes and per-test execution duration. The user must provide:

- `-program-results <file>` – path to a JSON file where all collected program-level execution data will be written.

An optional argument:

- `-timeout-min <seconds>` – sets a per-test timeout. If omitted, a default timeout is used. If set to a value ≤ 0, all timeout enforcement is disabled.

The resulting execution data are later used to optimise mutant execution and to support mutant classification.

2.1.2 Mutant mode

In mutant mode, the plugin executes a *single* mutant against the test suite. Test paths are supplied exactly as in regular Pytest usage. The user must again provide:

- `-program-results <file>` – read-only input containing the JSON data produced in program mode,
- `-mutant-results <file>` – path where execution results for the current mutant are written.

Optional arguments include:

- `-stop-on-kill`: stop executing further tests as soon as the mutant is killed.
- `-timeout-min <seconds>`: minimum allowable timeout.

- `-timeout-multiplier <float>`: defines the timeout for each test as:

  $timeout = \max(\texttt{timeout-min}, \texttt{timeout-multiplier} \times program_execution_time)$.

- `-disable-incompetent`: disables the classification of mutants as incompetent. By default, mutants may be labelled as *killed*, *survived*, *incompetent*, or *pending*.

mutant-sandbox-runner.py The script `mutant-sandbox-runner.py` orchestrates sandbox creation and iterated mutant execution. In other words, it reconstructs and executes multiple mutants sequentially or in parallel by internally invoking the `pytest-mutation-runner` plugin in *mutant* mode. This component expects the following obligatory arguments:

- `-base-repo`: root directory of the target project;
- `-src-path`: the Python module that is the target of mutation;
- `-tests`: test files or directories, provided exactly as they would be to Pytest;
- `-prog-res`: JSON file from which program-mode execution data will be read;
- `-mutants`: JSON file containing the mutant specifications needed to reconstruct mutated source code;
- `-save-path`: directory in which mutant-level result files will be written. For each mutant, one JSON file is produced as the output of internally running `pytest-mutation-runner` in mutant mode.

The component also inherits several optional arguments from `pytest-mutation-runner`, preserving their original semantics in mutant mode:

- `-stop-on-kill`,
- `-timeout-min`,
- `-timeout-multiplier`,
- `-disable-incompetent`.

In addition, the script introduces several optional arguments specific to its own functionality:

- `-njobs`: total number of jobs for partitioning the set of mutants (default: 1);

- `-jid`: job identifier in the range [0, `njobs` − 1] specifying which partition of the mutants to execute (default: 0). Together, `njobs` and `jid` support a static and nearly even division of mutants across parallel processes;

- `-mutant-ids`: optional list of specific mutant identifiers to execute, which overrides the partitioning logic based on `njobs`/`jid`;

- `-sandbox-mode`: selects the method for constructing the sandbox environment. The default mode, `copy`, creates a full copy of the repository and is suitable for typical use cases, including execution within WSL. The alternative mode, `symlink`, creates a lightweight sandbox using symbolic links and may be preferable for large repositories.

2.1.3 Notes

These components were implemented primarily for research purposes and are not intended to be production-ready. As such, the implementation makes certain assumptions, especially regarding corner cases, and expects that users will employ it in a reasonable way under the common scenarios for which it was designed. For example, it is assumed that the tests used to obtain the program execution results are the same tests used later during the mutant execution phase. Potential mismatches are neither expected nor checked in the code. Similar assumptions apply in other situations.

In some use cases, it may be beneficial for users to employ the implementation with additional care to remain on the safe side. For instance, it is good practice to configure a timeout that is higher than the default and at least a few seconds above the expected execution time of the tests. This allows for extra overhead, such as loading Pytest and the code under test, without causing unnecessary timeouts.

3 Mutant Propagation

Given a piece of code from a commit, a mutant, and its adjacent commit, our mutant mapper (implemented in `gumtree_mutant_mapper.py`) uses GumTree to determine whether the mutant can be transferred across commits. GumTree constructs ASTs for both versions of the code and produces an edit script that specifies the actions required to transform one AST into the other. The *Mutant Propagator* then processes GumTree's output to determine whether the mutant can be mapped to the adjacent commit and, if so, propagates the mutant accordingly.